

Biostat 203B Homework 5

Due Mar 20 @ 11:59PM

Nathaniel Boyle and 006525153

Table of contents

Predicting ICU duration	1
-----------------------------------	---

Predicting ICU duration

Using the ICU cohort `mimiciv_icu_cohort.rds` you built in Homework 4, develop at least three machine learning approaches (logistic regression with enet regularization, random forest, boosting, SVM, MLP, etc) plus a model stacking approach for predicting whether a patient's ICU stay will be longer than 2 days. You should use the `los_long` variable as the outcome. Your algorithms can use patient demographic information (gender, age at ICU `intime`, marital status, race), ICU admission information (first care unit), the last lab measurements before the ICU stay, and first vital measurements during ICU stay as features. You are welcome to use any feature engineering techniques you think are appropriate; but make sure to not use features that are not available at an ICU stay's `intime`. For instance, `last_careunit` cannot be used in your algorithms.

1. Data preprocessing and feature engineering.

Following slides on logistic regression

```
#Display system information for reproducibility
sessionInfo()
```

```
R version 4.4.2 (2024-10-31 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 26100)
```

```
Matrix products: default
```

```

locale:
[1] LC_COLLATE=English_United States.utf8
[2] LC_CTYPE=English_United States.utf8
[3] LC_MONETARY=English_United States.utf8
[4] LC_NUMERIC=C
[5] LC_TIME=English_United States.utf8

time zone: America/Los_Angeles
tzcode source: internal

attached base packages:
[1] stats      graphics  grDevices  utils      datasets  methods    base

loaded via a namespace (and not attached):
[1] compiler_4.4.2    fastmap_1.2.0     cli_3.6.4         tools_4.4.2
[5] htmltools_0.5.8.1 rstudioapi_0.17.1 yaml_2.3.10       tinytex_0.53
[9] rmarkdown_2.28    knitr_1.48        jsonlite_1.8.9    xfun_0.50
[13] digest_0.6.37     rlang_1.1.5       evaluate_1.0.3

```

```

# Installs
# Check if tidymodels is installed, and install if not

if (!requireNamespace("tidymodels", quietly = TRUE)) {
  install.packages("tidymodels")
}

# Check if rlang is installed, and install it if not
if (!requireNamespace("rlang", quietly = TRUE)) {
  install.packages("rlang")
}

# Check if cli is installed, and install it if not
if (!requireNamespace("cli", quietly = TRUE)) {
  install.packages("cli")
}

# Check if cli is installed, and install it if not
if (!requireNamespace("ranger", quietly = TRUE)) {
  install.packages("ranger")
}

# Check if cli is installed, and install it if not
if (!requireNamespace("keras", quietly = TRUE)) {

```

```

install.packages("keras")
}
# Check if cli is installed, and install it if not
if (!requireNamespace("stacks", quietly = TRUE)) {
  install.packages("stacks")
}

# Load libraries
library(GGally)

```

Loading required package: ggplot2

Registered S3 method overwritten by 'GGally':
 method from
 +.gg ggplot2

```

library(gtsummary)
library(tidyverse)

```

```

-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.0      v stringr    1.5.1
v lubridate  1.9.3      v tibble     3.2.1
v purrr      1.0.4      v tidyr      1.3.1

```

```

-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become

```

```

library(tidymodels)

```

```

-- Attaching packages ----- tidymodels 1.3.0 --
v broom      1.0.7      v rsample     1.2.1
v dials      1.4.0      v tune        1.3.0
v infer      1.0.7      v workflows   1.2.0
v modeldata  1.4.0      v workflowsets 1.1.0
v parsnip    1.3.0      v yardstick    1.3.2
v recipes    1.1.1

```

```
-- Conflicts ----- tidymodels_conflicts() --
x scales::discard() masks purrr::discard()
x dplyr::filter()   masks stats::filter()
x recipes::fixed()  masks stringr::fixed()
x dplyr::lag()       masks stats::lag()
x yardstick::spec() masks readr::spec()
x recipes::step()    masks stats::step()
```

```
library(dplyr)
library(forcats)
library(gtsummary)
library(ranger)
library(keras)
```

Attaching package: 'keras'

The following object is masked from 'package:yardstick':

```
get_weights
```

```
library(stacks)
```

```
version
```

```
platform      -
x86_64-w64-mingw32
arch          x86_64
os            mingw32
crt           ucrt
system        x86_64, mingw32
status
major         4
minor         4.2
year          2024
month         10
day           31
svn rev       87279
language      R
version.string R version 4.4.2 (2024-10-31 ucrt)
nickname      Pile of Leaves
```

```

# Load the mimic_icu_cohort data

mimiciv_icu_cohort <- readRDS("mimic_icu_cohort.rds") |>

# Remove columns not needed for analysis
select(-admit_provider_id, admittance, dischtime, deathtime, edregtime,
       edouttime) |>

# Lump infrequent levels into "Other" level for specific columns
# we take the top 5 most frequent values, rest are lumped into "other"
mutate(first_careunit = fct_lump_n(first_careunit, n = 5,
                                   other_level = "Other"),
       last_careunit = fct_lump_n(last_careunit, n = 5,
                                   other_level = "Other"),
       admission_type = fct_lump_n(admission_type, n = 5,
                                   other_level = "Other"),
       admission_location = fct_lump_n(admission_location, n = 5,
                                       other_level = "Other"),
       discharge_location = fct_lump_n(discharge_location, n = 5,
                                       other_level = "Other"),

# Collapse the levels of race into specified categories
race = fct_collapse(race,
                    ASIAN = c("ASIAN - SOUTH EAST ASIAN",
                              "ASIAN", "ASIAN - CHINESE",
                              "ASIAN - KOREAN",
                              "ASIAN - ASIAN INDIAN"),
                    BLACK = c("BLACK/AFRICAN AMERICAN",
                              "BLACK/CAPE VERDEAN",
                              "BLACK/AFRICAN",
                              "BLACK/CARIBBEAN ISLAND"),
                    HISPANIC = c("HISPANIC/LATINO - SALVADORAN",
                                  "HISPANIC/LATINO - PUERTO RICAN",
                                  "HISPANIC/LATINO - GUATEMALAN",
                                  "HISPANIC/LATINO - CUBAN",
                                  "HISPANIC/LATINO - DOMINICAN",
                                  "HISPANIC/LATINO - CENTRAL AMERICAN",
                                  "HISPANIC/LATINO - HONDURAN",
                                  "HISPANIC/LATINO - COLUMBIAN",
                                  "HISPANIC/LATINO - MEXICAN",
                                  "HISPANIC OR LATINO"),
                    WHITE = c("WHITE", "WHITE - RUSSIAN",

```

```

        "WHITE - OTHER EUROPEAN",
        "WHITE - BRAZILIAN",
        "WHITE - EASTERN EUROPEAN"),
    Other = c("OTHER", "UNKNOWN", "UNABLE TO OBTAIN",
        "PATIENT DECLINED TO ANSWER",
        "SOUTH AMERICAN",
        "MULTIPLE RACE/ETHNICITY",
        "AMERICAN INDIAN/ALASKA NATIVE",
        "NATIVE HAWAIIAN OR OTHER PACIFIC ISLANDER")),

    # Create a new variable `los_long` based on `los`
    los_long = los >= 2
)

# create summary (stratified by los_long)
summary_table <- mimiciu_icu_cohort |>
  tbl_summary(by = c("los_long"))

```

7 missing rows in the "los_long" column have been removed.

```

# Print the summary table
print(summary_table)

```

```

<div id="idesjosxyb" style="padding-left:0px;padding-right:0px;padding-top:10px;padding-bottom:10px">
  <table>#idesjosxyb table {
    font-family: system-ui, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif, 'Apple Color Emoji', 'Segoe UI Emoji', 'Segoe UI Symbol';
    -webkit-font-smoothing: antialiased;
    -moz-osx-font-smoothing: grayscale;
  }

  #idesjosxyb thead, #idesjosxyb tbody, #idesjosxyb tfoot, #idesjosxyb tr, #idesjosxyb td, #idesjosxyb th {
    border-style: none;
  }

  #idesjosxyb p {
    margin: 0;
    padding: 0;
  }

  #idesjosxyb .gt_table {
    display: table;
  }

```

```

border-collapse: collapse;
line-height: normal;
margin-left: auto;
margin-right: auto;
color: #333333;
font-size: 16px;
font-weight: normal;
font-style: normal;
background-color: #FFFFFF;
width: auto;
border-top-style: solid;
border-top-width: 2px;
border-top-color: #A8A8A8;
border-right-style: none;
border-right-width: 2px;
border-right-color: #D3D3D3;
border-bottom-style: solid;
border-bottom-width: 2px;
border-bottom-color: #A8A8A8;
border-left-style: none;
border-left-width: 2px;
border-left-color: #D3D3D3;
}

```

```

#idesjosxyb .gt_caption {
    padding-top: 4px;
    padding-bottom: 4px;
}

```

```

#idesjosxyb .gt_title {
    color: #333333;
    font-size: 125%;
    font-weight: initial;
    padding-top: 4px;
    padding-bottom: 4px;
    padding-left: 5px;
    padding-right: 5px;
    border-bottom-color: #FFFFFF;
    border-bottom-width: 0;
}

```

```

#idesjosxyb .gt_subtitle {
    color: #333333;
}

```

```

    font-size: 85%;
    font-weight: initial;
    padding-top: 3px;
    padding-bottom: 5px;
    padding-left: 5px;
    padding-right: 5px;
    border-top-color: #FFFFFF;
    border-top-width: 0;
}

#idesjosxyb .gt_heading {
    background-color: #FFFFFF;
    text-align: center;
    border-bottom-color: #FFFFFF;
    border-left-style: none;
    border-left-width: 1px;
    border-left-color: #D3D3D3;
    border-right-style: none;
    border-right-width: 1px;
    border-right-color: #D3D3D3;
}

#idesjosxyb .gt_bottom_border {
    border-bottom-style: solid;
    border-bottom-width: 2px;
    border-bottom-color: #D3D3D3;
}

#idesjosxyb .gt_col_headings {
    border-top-style: solid;
    border-top-width: 2px;
    border-top-color: #D3D3D3;
    border-bottom-style: solid;
    border-bottom-width: 2px;
    border-bottom-color: #D3D3D3;
    border-left-style: none;
    border-left-width: 1px;
    border-left-color: #D3D3D3;
    border-right-style: none;
    border-right-width: 1px;
    border-right-color: #D3D3D3;
}

```



```

#idesjosxyb .gt_col_heading {
    color: #333333;
    background-color: #FFFFFF;
    font-size: 100%;
    font-weight: normal;
    text-transform: inherit;
    border-left-style: none;
    border-left-width: 1px;
    border-left-color: #D3D3D3;
    border-right-style: none;
    border-right-width: 1px;
    border-right-color: #D3D3D3;
    vertical-align: bottom;
    padding-top: 5px;
    padding-bottom: 6px;
    padding-left: 5px;
    padding-right: 5px;
    overflow-x: hidden;
}

#idesjosxyb .gt_column_spanner_outer {
    color: #333333;
    background-color: #FFFFFF;
    font-size: 100%;
    font-weight: normal;
    text-transform: inherit;
    padding-top: 0;
    padding-bottom: 0;
    padding-left: 4px;
    padding-right: 4px;
}

#idesjosxyb .gt_column_spanner_outer:first-child {
    padding-left: 0;
}

#idesjosxyb .gt_column_spanner_outer:last-child {
    padding-right: 0;
}

#idesjosxyb .gt_column_spanner {
    border-bottom-style: solid;
    border-bottom-width: 2px;
}

```

```

border-bottom-color: #D3D3D3;
vertical-align: bottom;
padding-top: 5px;
padding-bottom: 5px;
overflow-x: hidden;
display: inline-block;
width: 100%;
}

#idesjosxyb .gt_spanner_row {
    border-bottom-style: hidden;
}

#idesjosxyb .gt_group_heading {
    padding-top: 8px;
    padding-bottom: 8px;
    padding-left: 5px;
    padding-right: 5px;
    color: #333333;
    background-color: #FFFFFF;
    font-size: 100%;
    font-weight: initial;
    text-transform: inherit;
    border-top-style: solid;
    border-top-width: 2px;
    border-top-color: #D3D3D3;
    border-bottom-style: solid;
    border-bottom-width: 2px;
    border-bottom-color: #D3D3D3;
    border-left-style: none;
    border-left-width: 1px;
    border-left-color: #D3D3D3;
    border-right-style: none;
    border-right-width: 1px;
    border-right-color: #D3D3D3;
    vertical-align: middle;
    text-align: left;
}

#idesjosxyb .gt_empty_group_heading {
    padding: 0.5px;
    color: #333333;
    background-color: #FFFFFF;

```

```

    font-size: 100%;
    font-weight: initial;
    border-top-style: solid;
    border-top-width: 2px;
    border-top-color: #D3D3D3;
    border-bottom-style: solid;
    border-bottom-width: 2px;
    border-bottom-color: #D3D3D3;
    vertical-align: middle;
}

#idesjosxyb .gt_from_md > :first-child {
    margin-top: 0;
}

#idesjosxyb .gt_from_md > :last-child {
    margin-bottom: 0;
}

#idesjosxyb .gt_row {
    padding-top: 8px;
    padding-bottom: 8px;
    padding-left: 5px;
    padding-right: 5px;
    margin: 10px;
    border-top-style: solid;
    border-top-width: 1px;
    border-top-color: #D3D3D3;
    border-left-style: none;
    border-left-width: 1px;
    border-left-color: #D3D3D3;
    border-right-style: none;
    border-right-width: 1px;
    border-right-color: #D3D3D3;
    vertical-align: middle;
    overflow-x: hidden;
}

#idesjosxyb .gt_stub {
    color: #333333;
    background-color: #FFFFFF;
    font-size: 100%;
    font-weight: initial;

```

```

    text-transform: inherit;
    border-right-style: solid;
    border-right-width: 2px;
    border-right-color: #D3D3D3;
    padding-left: 5px;
    padding-right: 5px;
}

#idesjosxyb .gt_stub_row_group {
    color: #333333;
    background-color: #FFFFFF;
    font-size: 100%;
    font-weight: initial;
    text-transform: inherit;
    border-right-style: solid;
    border-right-width: 2px;
    border-right-color: #D3D3D3;
    padding-left: 5px;
    padding-right: 5px;
    vertical-align: top;
}

#idesjosxyb .gt_row_group_first td {
    border-top-width: 2px;
}

#idesjosxyb .gt_row_group_first th {
    border-top-width: 2px;
}

#idesjosxyb .gt_summary_row {
    color: #333333;
    background-color: #FFFFFF;
    text-transform: inherit;
    padding-top: 8px;
    padding-bottom: 8px;
    padding-left: 5px;
    padding-right: 5px;
}

#idesjosxyb .gt_first_summary_row {
    border-top-style: solid;
    border-top-color: #D3D3D3;
}

```

```

}

#idesjosxyb .gt_first_summary_row.thick {
    border-top-width: 2px;
}

#idesjosxyb .gt_last_summary_row {
    padding-top: 8px;
    padding-bottom: 8px;
    padding-left: 5px;
    padding-right: 5px;
    border-bottom-style: solid;
    border-bottom-width: 2px;
    border-bottom-color: #D3D3D3;
}

#idesjosxyb .gt_grand_summary_row {
    color: #333333;
    background-color: #FFFFFF;
    text-transform: inherit;
    padding-top: 8px;
    padding-bottom: 8px;
    padding-left: 5px;
    padding-right: 5px;
}

#idesjosxyb .gt_first_grand_summary_row {
    padding-top: 8px;
    padding-bottom: 8px;
    padding-left: 5px;
    padding-right: 5px;
    border-top-style: double;
    border-top-width: 6px;
    border-top-color: #D3D3D3;
}

#idesjosxyb .gt_last_grand_summary_row_top {
    padding-top: 8px;
    padding-bottom: 8px;
    padding-left: 5px;
    padding-right: 5px;
    border-bottom-style: double;
    border-bottom-width: 6px;
}

```

```

    border-bottom-color: #D3D3D3;
}

#idesjosxyb .gt_stripped {
    background-color: rgba(128, 128, 128, 0.05);
}

#idesjosxyb .gt_table_body {
    border-top-style: solid;
    border-top-width: 2px;
    border-top-color: #D3D3D3;
    border-bottom-style: solid;
    border-bottom-width: 2px;
    border-bottom-color: #D3D3D3;
}

#idesjosxyb .gt_footnotes {
    color: #333333;
    background-color: #FFFFFF;
    border-bottom-style: none;
    border-bottom-width: 2px;
    border-bottom-color: #D3D3D3;
    border-left-style: none;
    border-left-width: 2px;
    border-left-color: #D3D3D3;
    border-right-style: none;
    border-right-width: 2px;
    border-right-color: #D3D3D3;
}

#idesjosxyb .gt_footnote {
    margin: 0px;
    font-size: 90%;
    padding-top: 4px;
    padding-bottom: 4px;
    padding-left: 5px;
    padding-right: 5px;
}

#idesjosxyb .gt_sourcenotes {
    color: #333333;
    background-color: #FFFFFF;
    border-bottom-style: none;

```

```

border-bottom-width: 2px;
border-bottom-color: #D3D3D3;
border-left-style: none;
border-left-width: 2px;
border-left-color: #D3D3D3;
border-right-style: none;
border-right-width: 2px;
border-right-color: #D3D3D3;
}

#idesjosxyb .gt_sourcenote {
    font-size: 90%;
    padding-top: 4px;
    padding-bottom: 4px;
    padding-left: 5px;
    padding-right: 5px;
}

#idesjosxyb .gt_left {
    text-align: left;
}

#idesjosxyb .gt_center {
    text-align: center;
}

#idesjosxyb .gt_right {
    text-align: right;
    font-variant-numeric: tabular-nums;
}

#idesjosxyb .gt_font_normal {
    font-weight: normal;
}

#idesjosxyb .gt_font_bold {
    font-weight: bold;
}

#idesjosxyb .gt_font_italic {
    font-style: italic;
}

```

```

#idesjosxyb .gt_super {
  font-size: 65%;
}

#idesjosxyb .gt_footnote_marks {
  font-size: 75%;
  vertical-align: 0.4em;
  position: initial;
}

#idesjosxyb .gt_asterisk {
  font-size: 100%;
  vertical-align: 0;
}

#idesjosxyb .gt_indent_1 {
  text-indent: 5px;
}

#idesjosxyb .gt_indent_2 {
  text-indent: 10px;
}

#idesjosxyb .gt_indent_3 {
  text-indent: 15px;
}

#idesjosxyb .gt_indent_4 {
  text-indent: 20px;
}

#idesjosxyb .gt_indent_5 {
  text-indent: 25px;
}

#idesjosxyb .katex-display {
  display: inline-flex !important;
  margin-bottom: 0.75em !important;
}

#idesjosxyb div.Reactable > div.rt-table > div.rt-thead > div.rt-tr.rt-tr-group-header > div
  height: 0px !important;
}

```



```

</style>
<table class="gt_table" data-quarto-disable-processing="false" data-quarto-bootstrap="false">
  <thead>
    <tr class="gt_col_headings">
      <th class="gt_col_heading gt_columns_bottom_border gt_left" rowspan="1" colspan="1" sc
      <th class="gt_col_heading gt_columns_bottom_border gt_center" rowspan="1" colspan="1" :
      <th class="gt_col_heading gt_columns_bottom_border gt_center" rowspan="1" colspan="1" :
    </tr>
  </thead>
  <tbody class="gt_table_body">
    <tr><td headers="label" class="gt_row gt_left">subject_id</td>
<td headers="stat_2" class="gt_row gt_center">15,023,390 (12,517,799, 17,521,365)</td>
<td headers="stat_1" class="gt_row gt_center">14,981,772 (12,502,618, 17,512,030)</td></tr>
    <tr><td headers="label" class="gt_row gt_left">hadm_id</td>
<td headers="stat_2" class="gt_row gt_center">25,011,923 (22,497,973, 27,472,462)</td>
<td headers="stat_1" class="gt_row gt_center">24,958,697 (22,470,100, 27,457,055)</td></tr>
    <tr><td headers="label" class="gt_row gt_left">stay_id</td>
<td headers="stat_2" class="gt_row gt_center">34,947,180 (32,472,773, 37,457,361)</td>
<td headers="stat_1" class="gt_row gt_center">35,045,524 (32,534,582, 37,514,057)</td></tr>
    <tr><td headers="label" class="gt_row gt_left">first_careunit</td>
<td headers="stat_2" class="gt_row gt_center"><br /></td>
<td headers="stat_1" class="gt_row gt_center"><br /></td></tr>
    <tr><td headers="label" class="gt_row gt_left">Cardiac Vascular Intensive Care Unit</td>
<td headers="stat_2" class="gt_row gt_center">7,350 (16%)</td>
<td headers="stat_1" class="gt_row gt_center">7,396 (16%)</td></tr>
    <tr><td headers="label" class="gt_row gt_left">Coronary Care Unit (CCU)</td>
<td headers="stat_2" class="gt_row gt_center">5,432 (12%)</td>
<td headers="stat_1" class="gt_row gt_center">5,278 (11%)</td></tr>
    <tr><td headers="label" class="gt_row gt_left">Medical Intensive Care Unit (MICU)</td>
<td headers="stat_2" class="gt_row gt_center">9,817 (21%)</td>
<td headers="stat_1" class="gt_row gt_center">10,729 (23%)</td></tr>
    <tr><td headers="label" class="gt_row gt_left">Medical/Surgical Intensive Care Unit</td>
<td headers="stat_2" class="gt_row gt_center">6,657 (14%)</td>
<td headers="stat_1" class="gt_row gt_center">8,664 (18%)</td></tr>
    <tr><td headers="label" class="gt_row gt_left">Surgical Intensive Care Unit (SICU)</td>
<td headers="stat_2" class="gt_row gt_center">6,428 (14%)</td>
<td headers="stat_1" class="gt_row gt_center">6,435 (14%)</td></tr>
    <tr><td headers="label" class="gt_row gt_left">Other</td>
<td headers="stat_2" class="gt_row gt_center">10,568 (23%)</td>
<td headers="stat_1" class="gt_row gt_center">8,901 (19%)</td></tr>
    <tr><td headers="label" class="gt_row gt_left">last_careunit</td>
<td headers="stat_2" class="gt_row gt_center"><br /></td>
<td headers="stat_1" class="gt_row gt_center"><br /></td></tr>

```

```
|  |
| --- |
| headers="label" class="gt_row gt_left">Cardiac Vascular Intensive Care Unit |
| headers="stat_2" class="gt_row gt_center">7,350 (16%)</td> |
| headers="stat_1" class="gt_row gt_center">7,396 (16%)</td></tr> |
| headers="label" class="gt_row gt_left">Coronary Care Unit (CCU)</td> |
| headers="stat_2" class="gt_row gt_center">5,432 (12%)</td> |
| headers="stat_1" class="gt_row gt_center">5,278 (11%)</td></tr> |
| headers="label" class="gt_row gt_left">Medical Intensive Care Unit (MICU)</td> |
| headers="stat_2" class="gt_row gt_center">9,817 (21%)</td> |
| headers="stat_1" class="gt_row gt_center">10,729 (23%)</td></tr> |
| headers="label" class="gt_row gt_left">Medical/Surgical Intensive Care Unit |
| headers="stat_2" class="gt_row gt_center">6,657 (14%)</td> |
| headers="stat_1" class="gt_row gt_center">8,664 (18%)</td></tr> |
| headers="label" class="gt_row gt_left">Surgical Intensive Care Unit (SICU)</td> |
| headers="stat_2" class="gt_row gt_center">6,428 (14%)</td> |
| headers="stat_1" class="gt_row gt_center">6,435 (14%)</td></tr> |
| headers="label" class="gt_row gt_left">Other</td> |
| headers="stat_2" class="gt_row gt_center">10,568 (23%)</td> |
| headers="stat_1" class="gt_row gt_center">8,901 (19%)</td></tr> |
| headers="label" class="gt_row gt_left">intime</td> |
| headers="stat_2" class="gt_row gt_center">2153-09-19 13:52:48 (2133-12-04 15:57:30, 2173 |
| headers="stat_1" class="gt_row gt_center">2153-10-01 01:04:01 (2133-11-05 07:59:46, 2173 |
| headers="label" class="gt_row gt_left">outtime</td> |
| headers="stat_2" class="gt_row gt_center">2153-09-24 03:45:31 (2133-12-12 21:11:09, 2173 |
| headers="stat_1" class="gt_row gt_center">2153-10-01 21:22:47 (2133-11-06 15:49:29, 2173 |
| headers="label" class="gt_row gt_left">los</td> |
| headers="stat_2" class="gt_row gt_center">3.9 (2.7, 6.8)</td> |
| headers="stat_1" class="gt_row gt_center">1.1 (0.8, 1.5)</td></tr> |
| headers="label" class="gt_row gt_left">admittime</td> |
| headers="stat_2" class="gt_row gt_center">2153-09-17 11:46:30 (2133-12-03 21:44:00, 2173 |
| headers="stat_1" class="gt_row gt_center">2153-09-30 07:15:00 (2133-11-02 14:47:00, 2173 |
| headers="label" class="gt_row gt_left">dischtime</td> |
| headers="stat_2" class="gt_row gt_center">2153-10-01 18:52:00 (2133-12-19 18:02:00, 2173 |
| headers="stat_1" class="gt_row gt_center">2153-10-07 02:00:00 (2133-11-11 14:34:00, 2173 |
| headers="label" class="gt_row gt_left">deathtime</td> |
| headers="stat_2" class="gt_row gt_center">2154-05-05 10:05:30 (2133-10-11 20:50:00, 2174 |
| headers="stat_1" class="gt_row gt_center">2153-08-12 12:35:00 (2133-08-21 14:08:00, 2173 |
| headers="label" class="gt_row gt_left">Unknown</td> |
| headers="stat_2" class="gt_row gt_center">39,456</td> |
| headers="stat_1" class="gt_row gt_center">43,254</td></tr> |
| headers="label" class="gt_row gt_left">admission_type</td> |
| headers="stat_2" class="gt_row gt_center"><br /></td> |
| headers="stat_1" class="gt_row gt_center"><br /></td></tr> |
| headers="label" class="gt_row gt_left">DIRECT EMER.</td> |

```

1,725 (3.7%)	
1,579 (3.3%)	
EW EMER.	
22,958 (50%)	
24,896 (53%)	
OBSERVATION ADMIT	
7,371 (16%)	
6,589 (14%)	
SURGICAL SAME DAY ADMISSION	
4,000 (8.6%)	
5,523 (12%)	
URGENT	
8,685 (19%)	
6,612 (14%)	
Other	
1,513 (3.3%)	
2,204 (4.6%)	
admission_location	
EMERGENCY ROOM	
17,038 (37%)	
20,058 (42%)	
PHYSICIAN REFERRAL	
11,001 (24%)	
12,590 (27%)	
TRANSFER FROM HOSPITAL	
13,875 (30%)	
10,283 (22%)	
TRANSFER FROM SKILLED NURSING FACILITY	
796 (1.7%)	
654 (1.4%)	
WALK-IN/SELF REFERRAL	
2,155 (4.7%)	
2,275 (4.8%)	
Other	
1,387 (3.0%)	
1,543 (3.3%)	
discharge_location	
DIED	
6,855 (15%)	

4,106 (8.8%)	
	HOME
6,871 (15%)	
15,060 (32%)	
	HOME HEALTH CARE
10,612 (23%)	
13,392 (29%)	
	REHAB
5,563 (12%)	
2,440 (5.2%)	
	SKILLED NURSING FACILITY
8,775 (19%)	
7,476 (16%)	
	Other
7,500 (16%)	
4,262 (9.1%)	
	Unknown
76	
667	
	insurance
	Medicaid
6,757 (15%)	
7,381 (16%)	
	Medicare
26,286 (58%)	
25,126 (54%)	
	No charge
5 (<0.1%)	
3 (<0.1%)	
	Other
1,087 (2.4%)	
1,178 (2.5%)	
	Private
11,491 (25%)	
12,875 (28%)	
	Unknown
626	
840	
	language

```
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| headers="label" class="gt_row gt_left"> American Sign Language</td>  headers="stat_2" class="gt_row gt_center">29 (<0.1%)</td>  headers="stat_1" class="gt_row gt_center">34 (<0.1%)</td></tr> |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | | headers="label" class="gt_row gt_left"> Amharic</td>  headers="stat_2" class="gt_row gt_center">14 (<0.1%)</td>  headers="stat_1" class="gt_row gt_center">9 (<0.1%)</td></tr> |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | | headers="label" class="gt_row gt_left"> Arabic</td>  headers="stat_2" class="gt_row gt_center">87 (0.2%)</td>  headers="stat_1" class="gt_row gt_center">62 (0.1%)</td></tr> |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | | headers="label" class="gt_row gt_left"> Armenian</td>  headers="stat_2" class="gt_row gt_center">12 (<0.1%)</td>  headers="stat_1" class="gt_row gt_center">11 (<0.1%)</td></tr> |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | | headers="label" class="gt_row gt_left"> Bengali</td>  headers="stat_2" class="gt_row gt_center">22 (<0.1%)</td>  headers="stat_1" class="gt_row gt_center">12 (<0.1%)</td></tr> |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | | headers="label" class="gt_row gt_left"> Chinese</td>  headers="stat_2" class="gt_row gt_center">550 (1.2%)</td>  headers="stat_1" class="gt_row gt_center">601 (1.3%)</td></tr> |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | | headers="label" class="gt_row gt_left"> English</td>  headers="stat_2" class="gt_row gt_center">41,487 (90%)</td>  headers="stat_1" class="gt_row gt_center">42,845 (91%)</td></tr> |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | | headers="label" class="gt_row gt_left"> French</td>  headers="stat_2" class="gt_row gt_center">18 (<0.1%)</td>  headers="stat_1" class="gt_row gt_center">14 (<0.1%)</td></tr> |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | | headers="label" class="gt_row gt_left"> Haitian</td>  headers="stat_2" class="gt_row gt_center">373 (0.8%)</td>  headers="stat_1" class="gt_row gt_center">252 (0.5%)</td></tr> |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | | headers="label" class="gt_row gt_left"> Hindi</td>  headers="stat_2" class="gt_row gt_center">24 (<0.1%)</td>  headers="stat_1" class="gt_row gt_center">21 (<0.1%)</td></tr> |  |  |  |  |  |  |  |  |  |  |  |  |  | | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | | headers="label" class="gt_row gt_left"> Italian</td>  headers="stat_2" class="gt_row gt_center">101 (0.2%)</td>  headers="stat_1" class="gt_row gt_center">106 (0.2%)</td></tr> |  |  |  |  |  |  |  |  |  |  | | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | | headers="label" class="gt_row gt_left"> Japanese</td>  headers="stat_2" class="gt_row gt_center">5 (<0.1%)</td>  headers="stat_1" class="gt_row gt_center">7 (<0.1%)</td></tr> |  |  |  |  |  |  |  | | --- | --- | --- | --- | --- | --- | --- | | headers="label" class="gt_row gt_left"> Kabuverdianu</td>  headers="stat_2" class="gt_row gt_center">301 (0.7%)</td>  headers="stat_1" class="gt_row gt_center">340 (0.7%)</td></tr> |  |  |  |  | | --- | --- | --- | --- | | headers="label" class="gt_row gt_left"> Khmer</td>  headers="stat_2" class="gt_row gt_center">50 (0.1%)</td>  headers="stat_1" class="gt_row gt_center">36 (<0.1%)</td></tr> |  | | --- | | headers="label" class="gt_row gt_left"> Korean</td> | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |

```

40 (<0.1%)
30 (<0.1%)
Modern Greek (1453-)
102 (0.2%)
85 (0.2%)
Other
152 (0.3%)
153 (0.3%)
Persian
42 (<0.1%)
35 (<0.1%)
Polish
36 (<0.1%)
37 (<0.1%)
Portuguese
350 (0.8%)
312 (0.7%)
Russian
599 (1.3%)
643 (1.4%)
Somali
8 (<0.1%)
15 (<0.1%)
Spanish
1,468 (3.2%)
1,413 (3.0%)
Thai
21 (<0.1%)
22 (<0.1%)
Vietnamese
151 (0.3%)
126 (0.3%)
Unknown
210
182
marital_status
DIVORCED
3,370 (8.0%)
3,522 (8.0%)
MARRIED
20,522 (49%)

```

<td headers="stat_1" class="gt_row gt_center">21,070 (48%)</td></tr>
  <tr><td headers="label" class="gt_row gt_left">    SINGLE</td>
<td headers="stat_2" class="gt_row gt_center">12,718 (30%)</td>
<td headers="stat_1" class="gt_row gt_center">13,834 (31%)</td></tr>
  <tr><td headers="label" class="gt_row gt_left">    WIDOWED</td>
<td headers="stat_2" class="gt_row gt_center">5,315 (13%)</td>
<td headers="stat_1" class="gt_row gt_center">5,664 (13%)</td></tr>
  <tr><td headers="label" class="gt_row gt_left">    Unknown</td>
<td headers="stat_2" class="gt_row gt_center">4,327</td>
<td headers="stat_1" class="gt_row gt_center">3,313</td></tr>
  <tr><td headers="label" class="gt_row gt_left">race</td>
<td headers="stat_2" class="gt_row gt_center"><br /></td>
<td headers="stat_1" class="gt_row gt_center"><br /></td></tr>
  <tr><td headers="label" class="gt_row gt_left">    Other</td>
<td headers="stat_2" class="gt_row gt_center">7,784 (17%)</td>
<td headers="stat_1" class="gt_row gt_center">6,553 (14%)</td></tr>
  <tr><td headers="label" class="gt_row gt_left">    ASIAN</td>
<td headers="stat_2" class="gt_row gt_center">1,367 (3.0%)</td>
<td headers="stat_1" class="gt_row gt_center">1,487 (3.1%)</td></tr>
  <tr><td headers="label" class="gt_row gt_left">    BLACK</td>
<td headers="stat_2" class="gt_row gt_center">4,920 (11%)</td>
<td headers="stat_1" class="gt_row gt_center">5,402 (11%)</td></tr>
  <tr><td headers="label" class="gt_row gt_left">    HISPANIC</td>
<td headers="stat_2" class="gt_row gt_center">1,682 (3.6%)</td>
<td headers="stat_1" class="gt_row gt_center">1,886 (4.0%)</td></tr>
  <tr><td headers="label" class="gt_row gt_left">    PORTUGUESE</td>
<td headers="stat_2" class="gt_row gt_center">234 (0.5%)</td>
<td headers="stat_1" class="gt_row gt_center">191 (0.4%)</td></tr>
  <tr><td headers="label" class="gt_row gt_left">    WHITE</td>
<td headers="stat_2" class="gt_row gt_center">30,265 (65%)</td>
<td headers="stat_1" class="gt_row gt_center">31,884 (67%)</td></tr>
  <tr><td headers="label" class="gt_row gt_left">edregtime</td>
<td headers="stat_2" class="gt_row gt_center">2154-06-26 08:30:00 (2134-12-30 15:10:00, 2174-
<td headers="stat_1" class="gt_row gt_center">2154-09-22 11:57:00 (2134-12-14 20:54:00, 2175-
  <tr><td headers="label" class="gt_row gt_left">    Unknown</td>
<td headers="stat_2" class="gt_row gt_center">16,213</td>
<td headers="stat_1" class="gt_row gt_center">15,900</td></tr>
  <tr><td headers="label" class="gt_row gt_left">edouttime</td>
<td headers="stat_2" class="gt_row gt_center">2154-06-27 00:03:00 (2134-12-30 19:14:00, 2174-
<td headers="stat_1" class="gt_row gt_center">2154-09-22 19:45:00 (2134-12-15 05:46:00, 2175-
  <tr><td headers="label" class="gt_row gt_left">    Unknown</td>
<td headers="stat_2" class="gt_row gt_center">16,213</td>
<td headers="stat_1" class="gt_row gt_center">15,900</td></tr>

```

```

        <tr><td headers="label" class="gt_row gt_left">hospital_expire_flag</td>
<td headers="stat_2" class="gt_row gt_center">6,803 (15%)</td>
<td headers="stat_1" class="gt_row gt_center">4,151 (8.8%)</td></tr>
        <tr><td headers="label" class="gt_row gt_left">gender</td>
<td headers="stat_2" class="gt_row gt_center"><br /></td>
<td headers="stat_1" class="gt_row gt_center"><br /></td></tr>
        <tr><td headers="label" class="gt_row gt_left">    F</td>
<td headers="stat_2" class="gt_row gt_center">20,070 (43%)</td>
<td headers="stat_1" class="gt_row gt_center">21,129 (45%)</td></tr>
        <tr><td headers="label" class="gt_row gt_left">    M</td>
<td headers="stat_2" class="gt_row gt_center">26,182 (57%)</td>
<td headers="stat_1" class="gt_row gt_center">26,274 (55%)</td></tr>
        <tr><td headers="label" class="gt_row gt_left">anchor_age</td>
<td headers="stat_2" class="gt_row gt_center">65 (54, 76)</td>
<td headers="stat_1" class="gt_row gt_center">64 (52, 75)</td></tr>
        <tr><td headers="label" class="gt_row gt_left">anchor_year</td>
<td headers="stat_2" class="gt_row gt_center">2,151 (2,132, 2,172)</td>
<td headers="stat_1" class="gt_row gt_center">2,152 (2,132, 2,172)</td></tr>
        <tr><td headers="label" class="gt_row gt_left">anchor_year_group</td>
<td headers="stat_2" class="gt_row gt_center"><br /></td>
<td headers="stat_1" class="gt_row gt_center"><br /></td></tr>
        <tr><td headers="label" class="gt_row gt_left">    2008 - 2010</td>
<td headers="stat_2" class="gt_row gt_center">13,945 (30%)</td>
<td headers="stat_1" class="gt_row gt_center">15,831 (33%)</td></tr>
        <tr><td headers="label" class="gt_row gt_left">    2011 - 2013</td>
<td headers="stat_2" class="gt_row gt_center">9,158 (20%)</td>
<td headers="stat_1" class="gt_row gt_center">10,151 (21%)</td></tr>
        <tr><td headers="label" class="gt_row gt_left">    2014 - 2016</td>
<td headers="stat_2" class="gt_row gt_center">8,951 (19%)</td>
<td headers="stat_1" class="gt_row gt_center">9,045 (19%)</td></tr>
        <tr><td headers="label" class="gt_row gt_left">    2017 - 2019</td>
<td headers="stat_2" class="gt_row gt_center">8,268 (18%)</td>
<td headers="stat_1" class="gt_row gt_center">7,621 (16%)</td></tr>
        <tr><td headers="label" class="gt_row gt_left">    2020 - 2022</td>
<td headers="stat_2" class="gt_row gt_center">5,930 (13%)</td>
<td headers="stat_1" class="gt_row gt_center">4,755 (10%)</td></tr>
        <tr><td headers="label" class="gt_row gt_left">dod</td>
<td headers="stat_2" class="gt_row gt_center">2155-09-06 (2135-07-16, 2175-10-08)</td>
<td headers="stat_1" class="gt_row gt_center">2156-01-17 (2136-06-15, 2176-07-11)</td></tr>
        <tr><td headers="label" class="gt_row gt_left">    Unknown</td>
<td headers="stat_2" class="gt_row gt_center">25,815</td>
<td headers="stat_1" class="gt_row gt_center">30,370</td></tr>
        <tr><td headers="label" class="gt_row gt_left">age</td>

```



```

<td headers="stat_2" class="gt_row gt_center">67 (56, 77)</td>
<td headers="stat_1" class="gt_row gt_center">66 (54, 77)</td></tr>
  <tr><td headers="label" class="gt_row gt_left">valuenum_avg_x</td>
<td headers="stat_2" class="gt_row gt_center">47 (41, 57)</td>
<td headers="stat_1" class="gt_row gt_center">45 (40, 52)</td></tr>
  <tr><td headers="label" class="gt_row gt_left">    Unknown</td>
<td headers="stat_2" class="gt_row gt_center">1</td>
<td headers="stat_1" class="gt_row gt_center">14</td></tr>
  <tr><td headers="label" class="gt_row gt_left">valuenum_avg_y</td>
<td headers="stat_2" class="gt_row gt_center">49 (42, 61)</td>
<td headers="stat_1" class="gt_row gt_center">43 (38, 50)</td></tr>
  <tr><td headers="label" class="gt_row gt_left">    Unknown</td>
<td headers="stat_2" class="gt_row gt_center">0</td>
<td headers="stat_1" class="gt_row gt_center">1</td></tr>
</tbody>

<tfoot class="gt_footnotes">
  <tr>
    <td class="gt_footnote" colspan="3"><span class="gt_footnote_marks" style="white-space
  </tr>
</tfoot>
</table>
</div>

```

2. Partition data into 50% training set and 50% test set. Stratify partitioning according to `los_long`. For grading purpose, sort the data by `subject_id`, `hadm_id`, and `stay_id` and use the seed 203 for the initial data split. Below is the sample code.

```

set.seed(203)

#Sort the data by subject_id, hadm_id, and stay_id
mimiciv_icu_cohort <- mimiciv_icu_cohort |>
  arrange(subject_id, hadm_id, stay_id) |>
  select(-c(intime, outtime, admittime, disctime, deathtime, edregtime,
            edouttime, dod, subject_id, hadm_id, stay_id, anchor_age))

# Drop rows with any NA values
mimiciv_icu_cohort <- mimiciv_icu_cohort |>
  drop_na()

# Now proceed with the rest of your analysis, such as recipe, model fitting, etc.

```

```
# Now proceed with the rest of your analysis, such as recipe, model fitting, etc.
```

```
# Perform the initial split (stratifying by los_long)
data_split <- initial_split(
  mimiciv_icu_cohort,
  strata = "los_long",
  prop = 0.1 #smaller training for faster code | change later
)
```

```
#check training output
mimiciv_other <- training(data_split)
dim(mimiciv_other)
```

```
[1] 8406    18
```

```
# Check for NAs in your dataset
anyNA(mimiciv_icu_cohort)
```

```
[1] FALSE
```

```
# Get the count of missing values by column
colSums(is.na(mimiciv_icu_cohort))
```

first_careunit	last_careunit	los
0	0	0
admission_type	admission_location	discharge_location
0	0	0
insurance	language	marital_status
0	0	0
race	hospital_expire_flag	gender
0	0	0
anchor_year	anchor_year_group	age
0	0	0
valuenum_avg_x	valuenum_avg_y	los_long
0	0	0

```
#check test output
mimiciv_test <- training(data_split)
dim(mimiciv_test)
```

```
[1] 8406    18
```

```
colSums(is.na(mimiciv_icu_cohort))
```

```

first_careunit      last_careunit      los
              0              0          0
admission_type    admission_location    discharge_location
              0              0          0
      insurance      language      marital_status
              0              0          0
      race hospital_expire_flag      gender
              0              0          0
anchor_year    anchor_year_group      age
              0              0          0
valuenum_avg_x    valuenum_avg_y    los_long
              0              0          0

```

3. Train and tune the models using the training set.

```
# Logistic Regression recipe
logit_recipe <- recipe(
  los_long ~ ., # Stratify on los_long
  data = mimiciv_other
) |>
# Handle new levels in categorical variables (e.g., language)
step_novel(all_nominal_predictors()) |>
# Convert 'los_long' to factor (if it isn't already a factor)
step_mutate(los_long = as.factor(los_long)) |>
# Impute missing numeric values with the median
step_impute_median(all_numeric_predictors()) |>
# Impute missing categorical values with the mode
step_impute_mode(all_nominal_predictors()) |>
# Convert all categorical predictors to dummy variables
step_dummy(all_nominal_predictors()) |>
# Remove zero-variance predictors
step_zv(all_numeric_predictors()) |>
# Normalize numeric predictors
```

```
step_normalize(all_numeric_predictors())  
  
# Optionally, you can inspect the recipe  
logit_recipe |> print()
```

```
-- Recipe -----
```

```
-- Inputs
```

```
Number of variables by role
```

```
outcome:    1  
predictor: 17
```

```
-- Operations
```

```
* Novel factor level assignment for: all_nominal_predictors()  
  
* Variable mutation for: as.factor(los_long)  
  
* Median imputation for: all_numeric_predictors()  
  
* Mode imputation for: all_nominal_predictors()  
  
* Dummy variables from: all_nominal_predictors()  
  
* Zero variance filter on: all_numeric_predictors()  
  
* Centering and scaling for: all_numeric_predictors()
```

```
# # Check if there are any remaining missing values in the dataset
anyNA(mimiciv_other)
```

```
[1] FALSE
```

```
table(mimiciv_other$los_long)
```

```
FALSE  TRUE
 4282  4124
```

```
# Logistic Regression Model

logit_mod <-
  logistic_reg(
    penalty = tune(),
    mixture = tune()
  ) |>
  set_engine("glmnet", standardize = FALSE) |>
  print()
```

Logistic Regression Model Specification (classification)

Main Arguments:

```
penalty = tune()
mixture = tune()
```

Engine-Specific Arguments:

```
standardize = FALSE
```

Computational engine: glmnet

```
#Set Logistic Workflow
logit_wf <- workflow() |>
  add_recipe(logit_recipe) |>
  add_model(logit_mod) |>
  print()
```

```

== Workflow =====
Preprocessor: Recipe
Model: logistic_reg()

-- Preprocessor -----
7 Recipe Steps

* step_novel()
* step_mutate()
* step_impute_median()
* step_impute_mode()
* step_dummy()
* step_zv()
* step_normalize()

-- Model -----
Logistic Regression Model Specification (classification)

Main Arguments:
  penalty = tune()
  mixture = tune()

Engine-Specific Arguments:
  standardize = FALSE

Computational engine: glmnet

```

```

# Create grid, tune penalty and mixture hyperparameters
param_grid <- grid_regular(
  penalty(range = c(-6, 3)),
  mixture(),
  levels = c(100, 5)
) |>
print()

```

```

# A tibble: 500 x 2
  penalty mixture
  <dbl>    <dbl>
1 0.000001      0
2 0.00000123    0
3 0.00000152    0
4 0.00000187    0

```

```

5 0.00000231      0
6 0.00000285      0
7 0.00000351      0
8 0.00000433      0
9 0.00000534      0
10 0.00000658     0
# i 490 more rows

```

```

# Cross Validation Set up
folds <- vfold_cv(mimiciv_other, v = 2)
folds

```

```

# 2-fold cross-validation
# A tibble: 2 x 2
  splits      id
  <list>     <chr>
1 <split [4203/4203]> Fold1
2 <split [4203/4203]> Fold2

```

```

#fit cross-validation

(logit_fit <- logit_wf |>
  tune_grid(
    resamples = folds,
    grid = param_grid,
    control = control_stack_grid(), #for stacking
    metrics = metric_set(roc_auc, accuracy)
  )) |>
  system.time()

```

```

New names:
New names:
New names:
New names:
New names:
New names:
New names:
New names:
New names:
New names:

```

New names:

```
* `anchor_year_group_X2011...2013` -> `anchor_year_group_X2011`  
* `anchor_year_group_X2014...2016` -> `anchor_year_group_X2014`  
* `anchor_year_group_X2017...2019` -> `anchor_year_group_X2017`  
* `anchor_year_group_X2020...2022` -> `anchor_year_group_X2020`
```

```
      user  system elapsed  
13.80    2.02    15.97
```

logit_fit

```
# Tuning results  
# 2-fold cross-validation  
# A tibble: 2 x 5  
  splits          id  .metrics          .notes          .predictions  
  <list>         <chr> <list>          <list>          <list>  
1 <split [4203/4203]> Fold1 <tibble [1,000 x 6]> <tibble [0 x 3]> <tibble>  
2 <split [4203/4203]> Fold2 <tibble [1,000 x 6]> <tibble [0 x 3]> <tibble>
```

#vizualize Cross Validation results

```
logit_fit |>  
  # aggregate metrics from K folds  
  collect_metrics() |>  
  print(width = Inf) |>  
  filter(.metric == "roc_auc") |>  
  ggplot(mapping = aes(x = penalty, y = mean, color = factor(mixture))) +  
  geom_point() +  
  labs(x = "Penalty", y = "CV AUC") +  
  scale_x_log10()
```

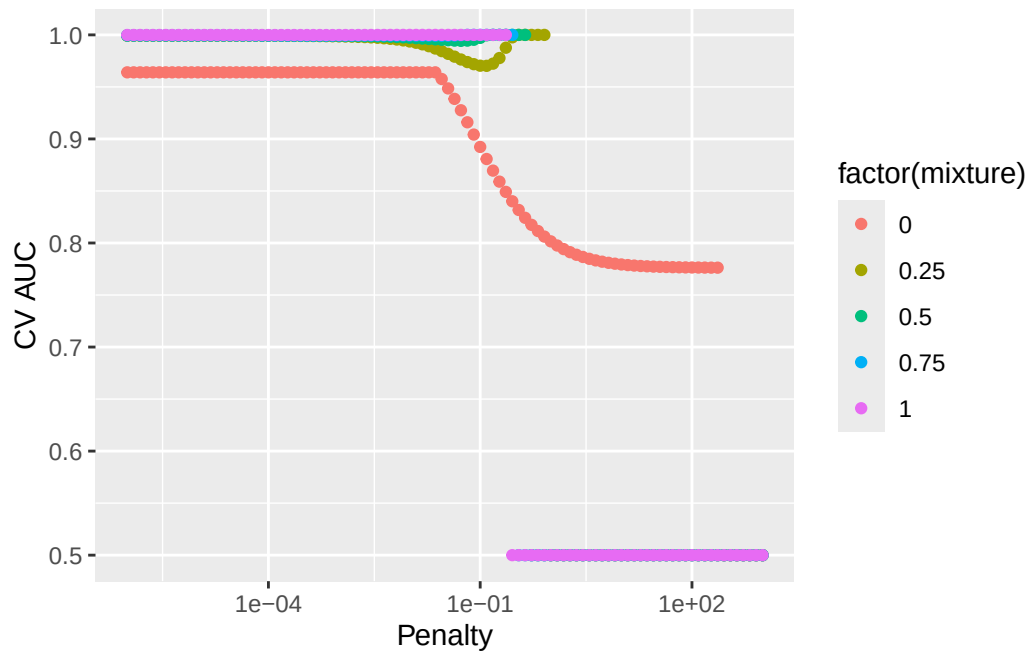
```
# A tibble: 1,000 x 8  
  penalty mixture .metric .estimator  mean     n std_err  
  <dbl>   <dbl> <chr>   <chr>    <dbl> <int>  <dbl>  
1 0.000001      0 accuracy binary    0.870     2 0.00654  
2 0.000001      0 roc_auc  binary    0.964     2 0.000687  
3 0.00000123     0 accuracy binary    0.870     2 0.00654  
4 0.00000123     0 roc_auc  binary    0.964     2 0.000687  
5 0.00000152     0 accuracy binary    0.870     2 0.00654  
6 0.00000152     0 roc_auc  binary    0.964     2 0.000687  
7 0.00000187     0 accuracy binary    0.870     2 0.00654
```



```

8 0.00000187      0 roc_auc  binary    0.964      2 0.000687
9 0.00000231      0 accuracy binary    0.870      2 0.00654
10 0.00000231     0 roc_auc  binary    0.964      2 0.000687
.config
<chr>
1 Preprocessor1_Model001
2 Preprocessor1_Model001
3 Preprocessor1_Model002
4 Preprocessor1_Model002
5 Preprocessor1_Model003
6 Preprocessor1_Model003
7 Preprocessor1_Model004
8 Preprocessor1_Model004
9 Preprocessor1_Model005
10 Preprocessor1_Model005
# i 990 more rows

```



Show top 5 LR models

```

logit_fit |>
  show_best(metric = "roc_auc")

```

```
# A tibble: 5 x 8
```

	penalty	mixture	.metric	.estimator	mean	n	std_err	.config
	<dbl>	<dbl>	<chr>	<chr>	<dbl>	<int>	<dbl>	<chr>
1	0.351	0.25	roc_auc	binary	1	2	0	Preprocessor1_Model162
2	0.433	0.25	roc_auc	binary	1	2	0	Preprocessor1_Model163
3	0.534	0.25	roc_auc	binary	1	2	0	Preprocessor1_Model164
4	0.658	0.25	roc_auc	binary	1	2	0	Preprocessor1_Model165
5	0.811	0.25	roc_auc	binary	1	2	0	Preprocessor1_Model166

Select best model

```
best_logit <- logit_fit |>
  select_best(metric = "roc_auc")
best_logit
```

```
# A tibble: 1 x 3
  penalty mixture .config
  <dbl>    <dbl> <chr>
1  0.351    0.25 Preprocessor1_Model162
```

Finalize LR model

```
# Final workflow
final_wf <- logit_wf |>
  finalize_workflow(best_logit)
final_wf
```

```
== Workflow =====
Preprocessor: Recipe
Model: logistic_reg()

-- Preprocessor -----
7 Recipe Steps

* step_novel()
* step_mutate()
* step_impute_median()
* step_impute_mode()
* step_dummy()
* step_zv()
* step_normalize()
```

```
-- Model -----  
Logistic Regression Model Specification (classification)
```

Main Arguments:

```
penalty = 0.351119173421513  
mixture = 0.25
```

Engine-Specific Arguments:

```
standardize = FALSE
```

Computational engine: glmnet

Fit the whole training set, predict test cases

```
# Fit the whole training set, then predict the test cases  
final_fit <-  
  final_wf |>  
  last_fit(data_split)
```

New names:

New names:

```
* `anchor_year_group_X2011...2013` -> `anchor_year_group_X2011`  
* `anchor_year_group_X2014...2016` -> `anchor_year_group_X2014`  
* `anchor_year_group_X2017...2019` -> `anchor_year_group_X2017`  
* `anchor_year_group_X2020...2022` -> `anchor_year_group_X2020`
```

```
final_fit
```

Resampling results

Manual resampling

A tibble: 1 x 6

	splits	id	.metrics	.notes	.predictions	.workflow
	<list>	<chr>	<list>	<list>	<list>	<list>
1	<split [8406/75657]>	train/test spl~	<tibble>	<tibble>	<tibble>	<workflow>

LR Test metrics

```
# Test metrics  
final_fit |>  
  collect_metrics()
```

```
# A tibble: 3 x 4
  .metric      .estimator .estimate .config
  <chr>        <chr>         <dbl> <chr>
1 accuracy    binary          0.739 Preprocessor1_Model1
2 roc_auc     binary          1      Preprocessor1_Model1
3 brier_class binary          0.218 Preprocessor1_Model1
```

We report that the Logistic regression model has an accuracy of 0.73 with an area under the ROC_AUC curve of 1.0

Training SVM Model

```
svm_recipe <- recipe(
  los_long ~ ., # Stratify on los_long
  data = mimiciv_other
) |>
# Convert 'los_long' to factor (if it isn't already a factor)
step_mutate(los_long = as.factor(los_long)) |>
# Handle new levels in categorical variables (e.g., language)
step_novel(all_nominal_predictors()) |>
# Impute missing values for numeric columns (mean imputation)
step_impute_mean(all_numeric_predictors()) |>
# Impute missing values for categorical/logical columns (mode imputation)
step_impute_mode(all_nominal_predictors()) |>
step_unknown(all_nominal_predictors()) |>
# Create dummy variables for nominal predictors
step_dummy(all_nominal_predictors()) |>
# Zero-variance filter for numeric predictors
step_zv(all_numeric_predictors()) |>
# Normalize numeric predictors
step_normalize(all_numeric_predictors())|>
print()
```

-- Recipe -----

-- Inputs

Number of variables by role

```
outcome:    1
predictor: 17
```

-- Operations

```
* Variable mutation for: as.factor(los_long)

* Novel factor level assignment for: all_nominal_predictors()

* Mean imputation for: all_numeric_predictors()

* Mode imputation for: all_nominal_predictors()

* Unknown factor level assignment for: all_nominal_predictors()

* Dummy variables from: all_nominal_predictors()

* Zero variance filter on: all_numeric_predictors()

* Centering and scaling for: all_numeric_predictors()
```

```
#SVM model classification
svm_mod <-
  svm_rbf(
    mode = "classification",
    cost = tune(),
    rbf_sigma = tune()
  ) |>
  set_engine("kernlab")
svm_mod
```

Radial Basis Function Support Vector Machine Model Specification (classification)

Main Arguments:

```
cost = tune()  
rbf_sigma = tune()
```

Computational engine: kernlab

SVM workflow

```
svm_wf <- workflow() |>  
  add_recipe(svm_recipe) |>  
  add_model(svm_mod)  
svm_wf
```

== Workflow =====

Preprocessor: Recipe

Model: svm_rbf()

-- Preprocessor -----

8 Recipe Steps

```
* step_mutate()  
* step_novel()  
* step_impute_mean()  
* step_impute_mode()  
* step_unknown()  
* step_dummy()  
* step_zv()  
* step_normalize()
```

-- Model -----

Radial Basis Function Support Vector Machine Model Specification (classification)

Main Arguments:

```
cost = tune()  
rbf_sigma = tune()
```

Computational engine: kernlab

SVM Tuning Grid

```
param_grid <- grid_regular(
  cost(range = c(-8, 5)),
  rbf_sigma(range = c(-5, -3)),
  levels = c(14, 5)
)
param_grid
```

```
# A tibble: 70 x 2
  cost rbf_sigma
  <dbl>   <dbl>
1 0.00391 0.00001
2 0.00781 0.00001
3 0.0156  0.00001
4 0.0312  0.00001
5 0.0625  0.00001
6 0.125   0.00001
7 0.25    0.00001
8 0.5     0.00001
9 1       0.00001
10 2      0.00001
# i 60 more rows
```

Cross-Validation

```
#install.packages("kernlab")
set.seed(203)

folds <- vfold_cv(mimiciv_other, v = 2)
folds
```

```
# 2-fold cross-validation
# A tibble: 2 x 2
  splits          id
  <list>         <chr>
1 <split [4203/4203]> Fold1
2 <split [4203/4203]> Fold2
```

```
#fit svm
svm_fit <- svm_wf |>
  tune_grid(
```

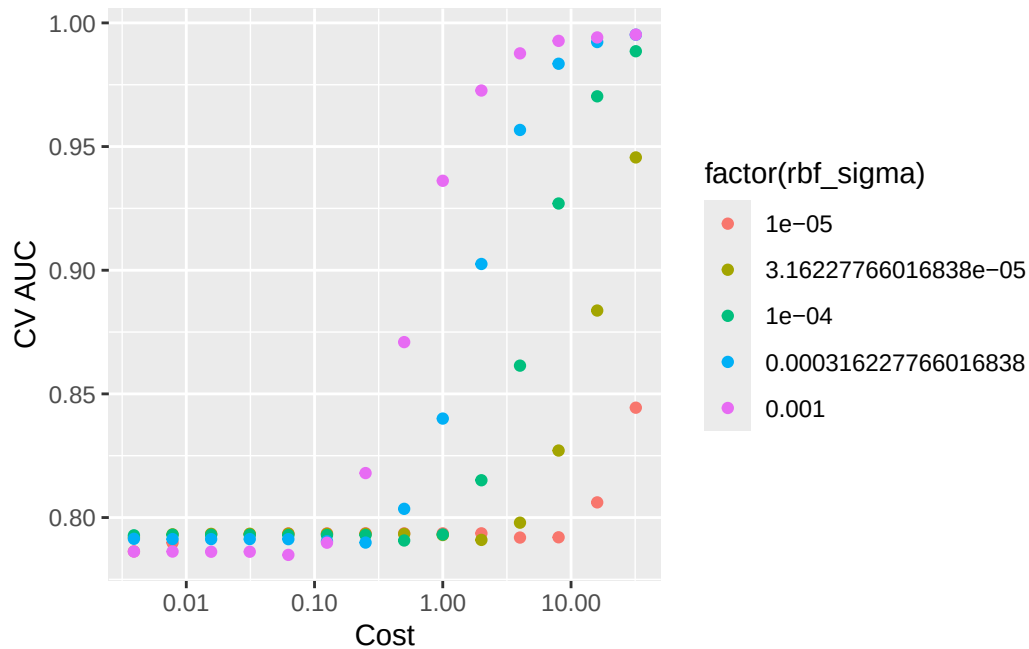

[illegible]

[illegible]


```
labs(x = "Cost", y = "CV AUC") +
scale_x_log10()
```

```
# A tibble: 140 x 8
```

	cost	rbf_sigma	.metric	.estimator	mean	n	std_err
	<dbl>	<dbl>	<chr>	<chr>	<dbl>	<int>	<dbl>
1	0.00391	0.00001	accuracy	binary	0.509	2	0.00309
2	0.00391	0.00001	roc_auc	binary	0.786	2	0.00484
3	0.00781	0.00001	accuracy	binary	0.509	2	0.00309
4	0.00781	0.00001	roc_auc	binary	0.790	2	0.00424
5	0.0156	0.00001	accuracy	binary	0.509	2	0.00309
6	0.0156	0.00001	roc_auc	binary	0.793	2	0.00574
7	0.0312	0.00001	accuracy	binary	0.509	2	0.00309
8	0.0312	0.00001	roc_auc	binary	0.793	2	0.00573
9	0.0625	0.00001	accuracy	binary	0.509	2	0.00309
10	0.0625	0.00001	roc_auc	binary	0.794	2	0.00575
.config							
	<chr>						
1	Preprocessor1_Model01						
2	Preprocessor1_Model01						
3	Preprocessor1_Model02						
4	Preprocessor1_Model02						
5	Preprocessor1_Model03						
6	Preprocessor1_Model03						
7	Preprocessor1_Model04						
8	Preprocessor1_Model04						
9	Preprocessor1_Model05						
10	Preprocessor1_Model05						
# i 130 more rows							



```
svm_fit |>
  show_best(metric = "roc_auc")
```

A tibble: 5 x 8

	cost	rbf_sigma	.metric	.estimator	mean	n	std_err	.config
	<dbl>	<dbl>	<chr>	<chr>	<dbl>	<int>	<dbl>	<chr>
1	32	0.001	roc_auc	binary	0.995	2	0.000261	Preprocessor1_Model170
2	32	0.000316	roc_auc	binary	0.995	2	0.000479	Preprocessor1_Model156
3	16	0.001	roc_auc	binary	0.994	2	0.0000385	Preprocessor1_Model169
4	8	0.001	roc_auc	binary	0.993	2	0.000475	Preprocessor1_Model168
5	16	0.000316	roc_auc	binary	0.992	2	0.00114	Preprocessor1_Model155

```
#select best model
best_svm <- svm_fit |>
  select_best(metric = "roc_auc")
best_svm
```

A tibble: 1 x 3

	cost	rbf_sigma	.config
	<dbl>	<dbl>	<chr>
1	32	0.001	Preprocessor1_Model170

Finalize SVM model

```
final_wf <- svm_wf |>
  finalize_workflow(best_svm)
final_wf
```

```
== Workflow =====
Preprocessor: Recipe
Model: svm_rbf()

-- Preprocessor -----
8 Recipe Steps

* step_mutate()
* step_novel()
* step_impute_mean()
* step_impute_mode()
* step_unknown()
* step_dummy()
* step_zv()
* step_normalize()

-- Model -----
Radial Basis Function Support Vector Machine Model Specification (classification)

Main Arguments:
  cost = 32
  rbf_sigma = 0.001

Computational engine: kernlab
```

```
# Fit the whole training set, then predict the test cases
final_fit <-
  final_wf |>
  last_fit(data_split)
```

New names:

New names:

```
* `anchor_year_group_X2011...2013` -> `anchor_year_group_X2011`
* `anchor_year_group_X2014...2016` -> `anchor_year_group_X2014`
* `anchor_year_group_X2017...2019` -> `anchor_year_group_X2017`
* `anchor_year_group_X2020...2022` -> `anchor_year_group_X2020`
```

```
final_fit
```

```
# Resampling results
# Manual resampling
# A tibble: 1 x 6
  splits          id      .metrics .notes  .predictions .workflow
  <list>         <chr>    <list>  <list>  <list>       <list>
1 <split [8406/75657]> train/test spl~ <tibble> <tibble> <tibble>   <workflow>
```

SVM test metrics

```
# Test metrics
final_fit |>
  collect_metrics()
```

```
# A tibble: 3 x 4
  .metric      .estimator .estimate .config
  <chr>        <chr>         <dbl>  <chr>
1 accuracy    binary         0.961  Preprocessor1_Model1
2 roc_auc     binary         0.997  Preprocessor1_Model1
3 brier_class binary         0.0216 Preprocessor1_Model1
```

For the SVM model we report an area under the roc_ curve of 0.99 with an accuracy of 96%

Train Random Forest Model

```
rf_recipe <- recipe(
  los_long ~ ., # Stratify on los_long
  data = mimiciv_other
) |>
# Convert 'los_long' to factor (if it isn't already a factor)
step_mutate(los_long = as.factor(los_long)) |>
# Handle new levels in categorical variables (e.g., language)
step_novel(all_nominal_predictors()) |>
# Impute missing values for numeric columns (mean imputation)
step_impute_mean(all_numeric_predictors()) |>
# Impute missing values for categorical/logical columns (mode imputation)
step_impute_mode(all_nominal_predictors()) |>
step_unknown(all_nominal_predictors()) |>
# Create dummy variables for nominal predictors
step_dummy(all_nominal_predictors()) |>
```

```
# Zero-variance filter for numeric predictors
step_zv(all_numeric_predictors()) |>
# Normalize numeric predictors
step_normalize(all_numeric_predictors())|>
print()
```

-- Recipe -----

-- Inputs

Number of variables by role

outcome: 1
predictor: 17

-- Operations

- * Variable mutation for: as.factor(los_long)
- * Novel factor level assignment for: all_nominal_predictors()
- * Mean imputation for: all_numeric_predictors()
- * Mode imputation for: all_nominal_predictors()
- * Unknown factor level assignment for: all_nominal_predictors()
- * Dummy variables from: all_nominal_predictors()
- * Zero variance filter on: all_numeric_predictors()
- * Centering and scaling for: all_numeric_predictors()


```
rf_mod <-
  rand_forest(
    mode = "classification",
    # Number of predictors randomly sampled in each split
    mtry = tune(),
    # Number of trees in ensemble
    trees = tune()
  ) |>
  set_engine("ranger")
rf_mod
```

Random Forest Model Specification (classification)

Main Arguments:

```
mtry = tune()
trees = tune()
```

Computational engine: ranger

```
rf_wf <- workflow() |>
  add_recipe(rf_recipe) |>
  add_model(rf_mod)
rf_wf
```

```
== Workflow =====
Preprocessor: Recipe
Model: rand_forest()

-- Preprocessor -----
8 Recipe Steps

* step_mutate()
* step_novel()
* step_impute_mean()
* step_impute_mode()
* step_unknown()
* step_dummy()
* step_zv()
* step_normalize()

-- Model -----
```

Random Forest Model Specification (classification)

Main Arguments:

```
mtry = tune()  
trees = tune()
```

Computational engine: ranger

```
#set grid  
param_grid <- grid_regular(  
  trees(range = c(100L, 500L)),  
  mtry(range = c(1L, 5L)),  
  levels = c(5, 5)  
)  
param_grid
```

```
# A tibble: 25 x 2  
  trees mtry  
  <int> <int>  
1   100     1  
2   200     1  
3   300     1  
4   400     1  
5   500     1  
6   100     2  
7   200     2  
8   300     2  
9   400     2  
10  500     2  
# i 15 more rows
```

```
#set folds  
set.seed(203)  
  
folds <- vfold_cv(mimiciv_other, v = 2)  
folds
```

```
# 2-fold cross-validation  
# A tibble: 2 x 2  
  splits id  
  <list> <chr>
```

```
1 <split [4203/4203]> Fold1
2 <split [4203/4203]> Fold2
```

```
#fit random forest model
rf_fit <- rf_wf |>
  tune_grid(
    resamples = folds,
    grid = param_grid,
    control = control_stack_grid(), #for stacking
    metrics = metric_set(roc_auc, accuracy)
  )
```

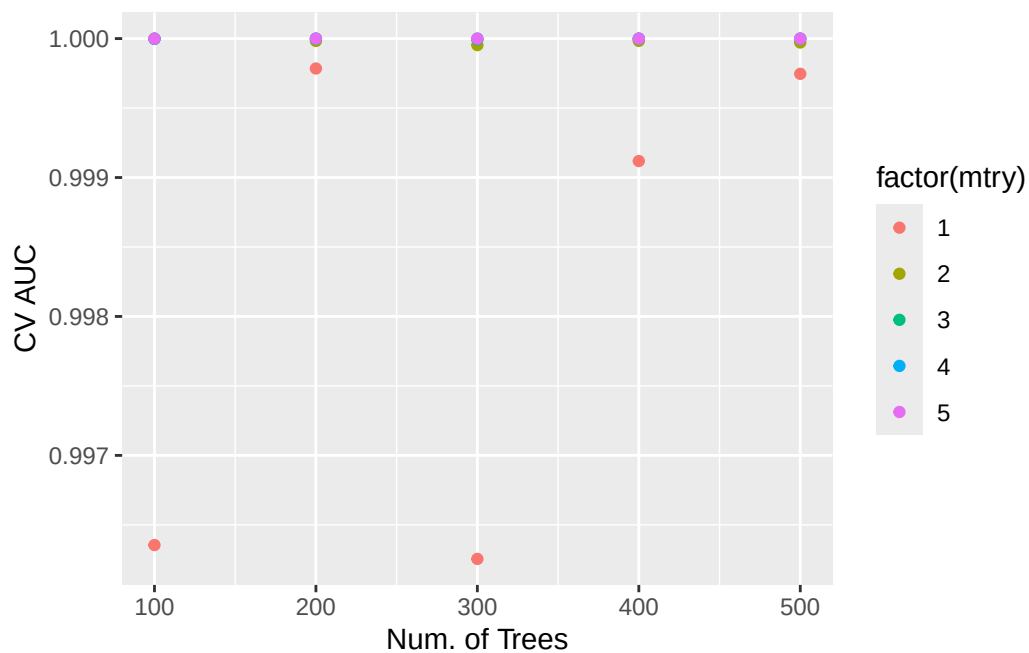
[illegible]


```
geom_point() +
# geom_line() +
labs(x = "Num. of Trees", y = "CV AUC")
```

A tibble: 50 x 8

	mtry	trees	.metric	.estimator	mean	n	std_err	.config
	<int>	<int>	<chr>	<chr>	<dbl>	<int>	<dbl>	<chr>
1	1	100	accuracy	binary	0.966	2	0.0236	Preprocessor1_Model101
2	1	100	roc_auc	binary	0.996	2	0.00320	Preprocessor1_Model101
3	1	200	accuracy	binary	0.985	2	0.00845	Preprocessor1_Model102
4	1	200	roc_auc	binary	1.00	2	0.0000397	Preprocessor1_Model102
5	1	300	accuracy	binary	0.960	2	0.00857	Preprocessor1_Model103
6	1	300	roc_auc	binary	0.996	2	0.000415	Preprocessor1_Model103
7	1	400	accuracy	binary	0.982	2	0.00119	Preprocessor1_Model104
8	1	400	roc_auc	binary	0.999	2	0.000490	Preprocessor1_Model104
9	1	500	accuracy	binary	0.989	2	0.00500	Preprocessor1_Model105
10	1	500	roc_auc	binary	1.00	2	0.00000594	Preprocessor1_Model105

i 40 more rows



Shoow top 5 models

```
rf_fit |>
  show_best(metric = "roc_auc")
```

```
# A tibble: 5 x 8
  mtry trees .metric .estimator mean     n   std_err .config
  <int> <int> <chr>   <chr>   <dbl> <int>   <dbl> <chr>
1     4    200 roc_auc binary    1.00     2 0.000000227 Preprocessor1_Model17
2     4    300 roc_auc binary    1.00     2 0.000000227 Preprocessor1_Model18
3     4    400 roc_auc binary    1.00     2 0.000000227 Preprocessor1_Model19
4     4    500 roc_auc binary    1.00     2 0.000000227 Preprocessor1_Model20
5     5    100 roc_auc binary    1.00     2 0.000000227 Preprocessor1_Model21
```

Choose best model

```
best_rf <- rf_fit |>
  select_best(metric = "roc_auc")
best_rf
```

```
# A tibble: 1 x 3
  mtry trees .config
  <int> <int> <chr>
1     4    200 Preprocessor1_Model17
```

Finalize RF model

```
# Final workflow
final_wf <- rf_wf |>
  finalize_workflow(best_rf)
final_wf
```

```
== Workflow =====
Preprocessor: Recipe
Model: rand_forest()

-- Preprocessor -----
8 Recipe Steps

* step_mutate()
* step_novel()
* step_impute_mean()
```

```

* step_impute_mode()
* step_unknown()
* step_dummy()
* step_zv()
* step_normalize()

```

```

-- Model -----
Random Forest Model Specification (classification)

```

Main Arguments:

```

  mtry = 4
  trees = 200

```

Computational engine: ranger

```

# Fit the whole training set, then predict the test cases
final_fit <-
  final_wf |>
  last_fit(data_split)

```

New names:

New names:

```

* `anchor_year_group_X2011...2013` -> `anchor_year_group_X2011`
* `anchor_year_group_X2014...2016` -> `anchor_year_group_X2014`
* `anchor_year_group_X2017...2019` -> `anchor_year_group_X2017`
* `anchor_year_group_X2020...2022` -> `anchor_year_group_X2020`

```

```

final_fit

```

Resampling results

Manual resampling

A tibble: 1 x 6

	splits	id	.metrics	.notes	.predictions	.workflow
	<list>	<chr>	<list>	<list>	<list>	<list>
1	<split [8406/75657]>	train/test spl~	<tibble>	<tibble>	<tibble>	<workflow>

Test metrics

```

final_fit |>
  collect_metrics()

```

```
# A tibble: 3 x 4
  .metric      .estimator .estimate .config
  <chr>        <chr>         <dbl> <chr>
1 accuracy    binary          1.00  Preprocessor1_Model1
2 roc_auc     binary          1.00  Preprocessor1_Model1
3 brier_class binary          0.0465 Preprocessor1_Model1
```

The RF model has the highest accuracy of 99% and an area under the ROC AUC curve of 1.

Stacking the models

Shared recipe

```
stacked_recipe <-
  recipe(
    los_long ~ .,
    data = mimiciv_other
  ) |>
  # Convert 'los_long' to factor (if it isn't already a factor)
  step_mutate(los_long = as.factor(los_long)) |>
  # Handle new levels in categorical variables (e.g., language)
  step_novel(all_nominal_predictors()) |>
  # Impute missing values for numeric columns (mean imputation)
  step_impute_mean(all_numeric_predictors()) |>
  # Impute missing values for categorical/logical columns (mode imputation)
  step_impute_mode(all_nominal_predictors()) |>
  step_unknown(all_nominal_predictors()) |>
  # Create dummy variables for nominal predictors
  step_dummy(all_nominal_predictors()) |>
  # Zero-variance filter for numeric predictors
  step_zv(all_numeric_predictors()) |>
  # Normalize numeric predictors
  step_normalize(all_numeric_predictors())
stacked_recipe
```

-- Recipe -----

-- Inputs

Number of variables by role

outcome: 1
predictor: 17

-- Operations

- * Variable mutation for: `as.factor(los_long)`
- * Novel factor level assignment for: `all_nominal_predictors()`
- * Mean imputation for: `all_numeric_predictors()`
- * Mode imputation for: `all_nominal_predictors()`
- * Unknown factor level assignment for: `all_nominal_predictors()`
- * Dummy variables from: `all_nominal_predictors()`
- * Zero variance filter on: `all_numeric_predictors()`
- * Centering and scaling for: `all_numeric_predictors()`

Logistic Regression Model

```
logit_mod <-  
  logistic_reg(  
    penalty = tune(),  
    mixture = tune()  
  ) |>  
  set_engine("glmnet", standardize = TRUE)  
logit_mod
```

Logistic Regression Model Specification (classification)

Main Arguments:

```
penalty = tune()  
mixture = tune()
```

Engine-Specific Arguments:

```
standardize = TRUE
```

Computational engine: glmnet

```
logit_wf <- workflow() |>  
  add_recipe(stacked_recipe) |>  
  add_model(logit_mod)  
logit_wf
```

== Workflow =====

Preprocessor: Recipe

Model: logistic_reg()

-- Preprocessor -----

8 Recipe Steps

```
* step_mutate()  
* step_novel()  
* step_impute_mean()  
* step_impute_mode()  
* step_unknown()  
* step_dummy()  
* step_zv()  
* step_normalize()
```

-- Model -----

Logistic Regression Model Specification (classification)

Main Arguments:

```
penalty = tune()  
mixture = tune()
```

Engine-Specific Arguments:

```
standardize = TRUE
```

Computational engine: glmnet

```
logit_grid <- grid_regular(  
  penalty(range = c(-6, 3)),  
  mixture(),  
  levels = c(100, 5)  
)  
  
logit_res <-  
  tune_grid(  
    object = logit_wf,  
    resamples = folds,  
    grid = logit_grid,  
    control = control_stack_grid()  
  )
```

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

* `anchor\_year\_group\_X2011...2013` -> `anchor\_year\_group\_X2011`

* `anchor\_year\_group\_X2014...2016` -> `anchor\_year\_group\_X2014`

* `anchor\_year\_group\_X2017...2019` -> `anchor\_year\_group\_X2017`

* `anchor\_year\_group\_X2020...2022` -> `anchor\_year\_group\_X2020`

logit_res

Tuning results

2-fold cross-validation

A tibble: 2 x 5

	splits	id	.metrics	.notes	.predictions
	<list>	<chr>	<list>	<list>	<list>
1	<split [4203/4203]>	Fold1	<tibble [1,500 x 6]>	<tibble [0 x 3]>	<tibble>
2	<split [4203/4203]>	Fold2	<tibble [1,500 x 6]>	<tibble [0 x 3]>	<tibble>

Random Forest Model

```
rf_mod <-  
  rand_forest(  
    mode = "classification",  
    # Number of predictors randomly sampled in each split  
    mtry = tune(),  
    # Number of trees in ensemble  
    trees = tune()  
  ) |>  
  set_engine("ranger")  
rf_mod
```

Random Forest Model Specification (classification)

Main Arguments:

```
mtry = tune()  
trees = tune()
```

Computational engine: ranger

```
rf_wf <- workflow() |>  
  add_recipe(stacked_recipe) |>  
  add_model(rf_mod)  
rf_wf
```

```
== Workflow =====  
Preprocessor: Recipe  
Model: rand_forest()  
  
-- Preprocessor -----  
8 Recipe Steps  
  
* step_mutate()  
* step_novel()  
* step_impute_mean()  
* step_impute_mode()  
* step_unknown()  
* step_dummy()  
* step_zv()  
* step_normalize()
```

-- Model -----
Random Forest Model Specification (classification)

Main Arguments:

```
mtry = tune()  
trees = tune()
```

Computational engine: ranger

```
rf_grid <- grid_regular(  
  trees(range = c(100L, 500L)),  
  mtry(range = c(1L, 5L)),  
  levels = c(5, 5)  
)  
  
rf_res <-  
  tune_grid(  
    object = rf_wf,  
    resamples = folds,  
    grid = rf_grid,  
    control = control_stack_grid()  
  )
```

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:

New names:


```
# 2-fold cross-validation
# A tibble: 2 x 5
  splits          id    .metrics          .notes          .predictions
  <list>         <chr> <list>          <list>          <list>
1 <split [4203/4203]> Fold1 <tibble [75 x 6]> <tibble [0 x 3]> <tibble>
2 <split [4203/4203]> Fold2 <tibble [75 x 6]> <tibble [0 x 3]> <tibble>
```

Stack Logistic Regression and Randomforest

```
stack_model <-
  # initialize the stack
  stacks() |>
  # add candidate members (fits)
  add_candidates(logit_res) |>
  add_candidates(rf_res) |>
  #add_candidates(mlp_res) |>
  # determine how to combine their predictions
  blend_predictions(
    penalty = 10^(-6:2),
    metrics = c("roc_auc")
  ) |>
  # fit the candidates with nonzero stacking coefficients
  fit_members()
```

New names:

New names:

New names:

New names:

New names:

New names:

New names:

* `anchor\_year\_group\_X2011...2013` -> `anchor\_year\_group\_X2011`

* `anchor\_year\_group\_X2014...2016` -> `anchor\_year\_group\_X2014`

* `anchor\_year\_group\_X2017...2019` -> `anchor\_year\_group\_X2017`

* `anchor\_year\_group\_X2020...2022` -> `anchor\_year\_group\_X2020`

Plot statistics of stack models.

```
stack_model
```

```
-- A stacked ensemble model -----
```

Out of 251 possible candidate members, the ensemble retained 7.

Penalty: 1e-06.

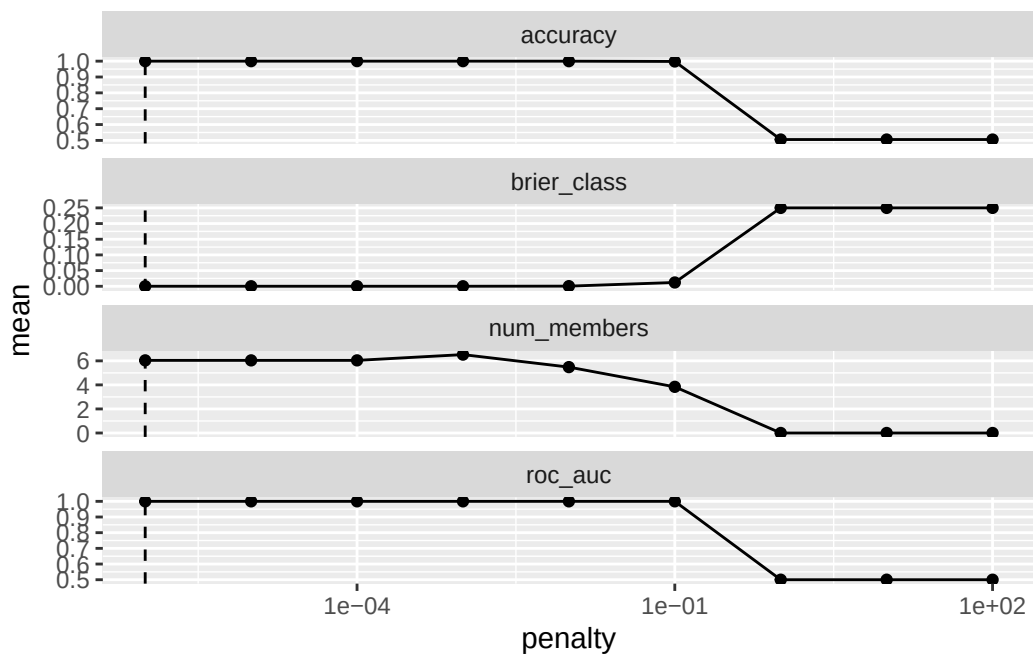
Mixture: 1.

The 7 highest weighted member classes are:

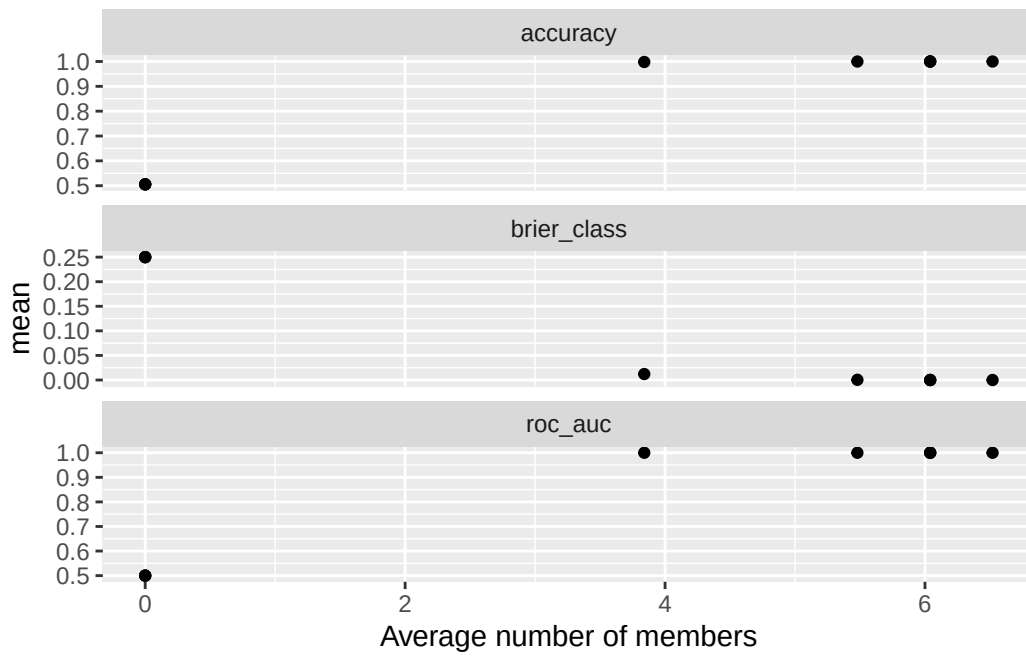
A tibble: 7 x 3

	member <chr>	type <chr>	weight <dbl>
1	.pred_TRUE_rf_res_1_21	rand_forest	15.7
2	.pred_TRUE_rf_res_1_24	rand_forest	6.35
3	.pred_TRUE_logit_res_1_401	logistic_reg	2.10
4	.pred_TRUE_rf_res_1_22	rand_forest	0.875
5	.pred_TRUE_logit_res_1_417	logistic_reg	0.0246
6	.pred_TRUE_logit_res_1_418	logistic_reg	0.00920
7	.pred_TRUE_logit_res_1_419	logistic_reg	0.00177

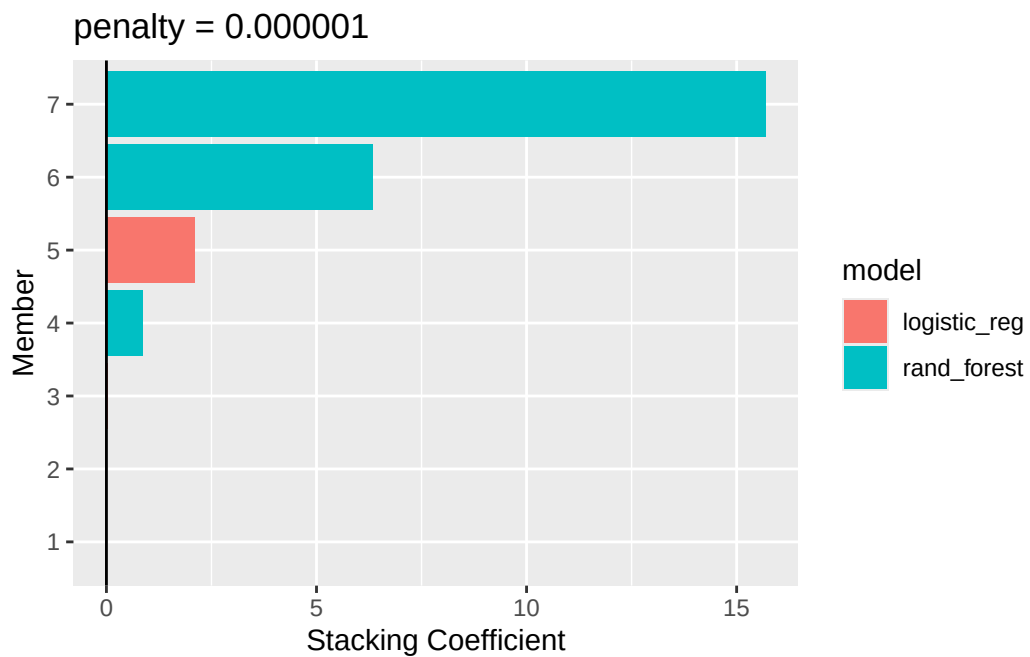
```
autoplot(stack_model)
```




```
autoplot(stack_model, type = "members")
```



```
autoplot(stack_model, type = "weights")
```



```
collect_parameters(stack_model, "rf_res")
```

```
# A tibble: 25 x 5
  member      mtry trees terms      coef
  <chr>    <int> <int> <chr>    <dbl>
1 rf_res_1_01      1  100 .pred_TRUE_rf_res_1_01      0
2 rf_res_1_02      1  200 .pred_TRUE_rf_res_1_02      0
3 rf_res_1_03      1  300 .pred_TRUE_rf_res_1_03      0
4 rf_res_1_04      1  400 .pred_TRUE_rf_res_1_04      0
5 rf_res_1_05      1  500 .pred_TRUE_rf_res_1_05      0
6 rf_res_1_06      2  100 .pred_TRUE_rf_res_1_06      0
7 rf_res_1_07      2  200 .pred_TRUE_rf_res_1_07      0
8 rf_res_1_08      2  300 .pred_TRUE_rf_res_1_08      0
9 rf_res_1_09      2  400 .pred_TRUE_rf_res_1_09      0
10 rf_res_1_10     2  500 .pred_TRUE_rf_res_1_10      0
# i 15 more rows
```

4. Compare model classification performance on the test set. Report both the area under ROC curve and accuracy for each machine learning algorithm and the model stacking. Interpret the results.

From the different models used we report that the random forest model was the best in terms of accuracy and area under the ROC AUC curve. There is certainly overfitting occurring in this situation however, and it is likely given the current tuning that applying this model to new datasets will likely result in poor performance.

What are the most important features in predicting long ICU stays? How do the models compare in terms of performance and interpretability?

The SVM model took by far the longest to train and to evaluate, estimating about 30 minutes of run time. The logistic regression model was by far the fastest and also had the best interpretability.